Building data course — Week 1 — find your way around (read-only)

sparql-playground / lessons

Generated for print and offline reading

Week 1 — find your way around (read-only)

Five daily lessons. Run the SQL in DB Browser for SQLite against the API's own database, `var/timeseries.sqlite` (build it with `uv run bldgts bootstrap`). Lines like `-- expect: 288` are the answer you should see. Keep notes under the ignored `student-work/` folder.

Contents

- Day 1 — Open the database; what one row is
- Day 2 — SELECT, WHERE, ORDER BY, LIMIT on one sensor; what the UUIDs are

- Day 3 — Time: epoch seconds, `datetime()`, date ranges
- Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM
- Day 5 — GROUP BY and HAVING: per day, per month, per series

---

Day 1 — Open the database; what one row is

Goal: open the API's database in DB Browser for SQLite and say in one sentence what a row means.

Read

An HVAC controller measures things (temperatures, pressures, fan status) and a **historian** saves a reading every few minutes. Building 50's historian exported a CSV with one column per point. `uv run bldgts bootstrap` turned that into the API's SQLite database, `var/timeseries.sqlite`.

SQLite is a whole SQL database in one file: no server, no login. Python ships with it (`import sqlite3`), and so do phones and browsers.

Open it **read-only** this week, so nothing you click can change it:

1. Start DB Browser for SQLite.
2. Click the small arrow next to **Open Database** and choose **Open Database Read Only**.
3. Pick `var/timeseries.sqlite` in this repo.

On the **Database Structure** tab you'll see two tables:

Table	What it holds
`samples`	every reading: `timeseries_id`, `ts`, `value`
`ingest_log`	one row per CSV the bootstrap loaded (file hash and row count), so it can skip unchanged files

Click the arrow next to `samples` to see its columns and the `CREATE TABLE` statement:

```
CREATE TABLE samples (
    timeseries_id TEXT NOT NULL,
    ts INTEGER NOT NULL,
    value REAL,
    PRIMARY KEY (timeseries_id, ts)
) WITHOUT ROWID
```

Read it like this: - **One row is one reading:** this point (`timeseries_id`) had this `value` at this time (`ts`). - The **primary key** `(timeseries_id, ts)` means a point can't have

two readings at the same moment. - `WITHOUT ROWID` stores rows sorted by that key, so all the readings of one point sit together, in time order. Day 12 shows why that makes reads fast.

This shape is called **long** (or narrow) format: three columns, millions of rows. The CSV it came from was **wide**: one row per timestamp and one column per point. Day 11 converts between the two.

Now open **Browse Data**, pick `samples`, and scroll. Then go to **Execute SQL**, paste a query, and press **Ctrl+Return** (or F5) to run it.

Run

```sql
-- How many readings are there?
SELECT count(*) FROM samples;
-- expect: 4193248


-- The first five rows, in key order
SELECT * FROM samples LIMIT 5;
-- expect rows: 5


-- What did the bootstrap load?
SELECT dataset_key, row_count FROM
         ingest_log;
-- expect rows: 2
```

The two `row_counts` (2,949,488 for AHU_1 and 1,243,760 for AHU_2) add up to the total above.

Try it

1. In **Browse Data**, filter `timeseries_id` by typing the first few characters of any id. Do all its rows share the same id and step forward in `ts`?
2. How many different points are there?

Answer

```
SELECT count(DISTINCT timeseries_id) FROM
         samples;
-- expect: 118
```

118 points (series). 4,193,248 / 118 = 35,536 readings each: every point was logged at every timestamp.

Why it matters

Every time-series database (Timescale, InfluxDB, a data lake) stores some version of *(series id, time, value)*. The hard part is never storing it. It's knowing **which** series id is the duct static pressure on AHU_1. That knowledge isn't in this table at all, and finding it is what the RDF half of the course is about.

Day 2 — SELECT, WHERE, ORDER BY, LIMIT on one sensor; what the UUIDs are

Goal: pull the readings of one sensor, sorted and trimmed, and know where its id comes from.

Read

A `timeseries_id` looks like `bf422925-1e2d-598d-9069-3b0f17d49c48`. That's a **UUID**, a 128-bit id written as 32 hex digits. These are version-5 UUIDs: a hash of a name, so the same building/point always gets the same UUID. They don't change between rebuilds, and the API can hand them out without exposing the BAS's internal point names.

The database doesn't know what a UUID means. That lives in the point list the model is built from, `model/points/BUILDING_50__AHU_1.csv`. Open it in a text editor or spreadsheet and find this row:

source_column	point_id	b
outside_air_temp_f	AHU_1_OA_T	Outside_Air

source_column	point_id	b

So `bf422925…` is AHU_1's outside air temperature in °F. Here are a few you'll use this week:

Point	timeseries_id
AHU_1 outside air temp (°F)	`bf422925-1e2d-598d-9069-3b0f17d49c48`
AHU_1 supply fan status (1 = running)	`9ebf80dd-7821-510b-b2c0-cd0151e093c9`
AHU_1 duct static pressure (inH2O)	`52422584-9894-50bd-9696-2fbb8b062518`
AHU_1 duct static pressure setpoint (inH2O)	`761c2b61-4982-5157-bc66-2010cf6e2a47`

The parts of a query, in the order you write them:

```
SELECT  columns            -- what to show
FROM    table              -- where from
WHERE   condition          -- which rows
ORDER BY column [DESC]     -- sorted how
LIMIT   n                  -- how many
```

Text values go in single quotes. Double quotes are for column and table names.

Run

```
-- The first three outside air readings
SELECT ts, value
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
          9069-3b0f17d49c48'
ORDER BY ts
LIMIT 3;
-- expect: 1773621600 | 69.0327


-- The hottest reading
SELECT ts, value
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
          9069-3b0f17d49c48'
ORDER BY value DESC
LIMIT 1;
-- expect: 1784065200 | 98.8608


-- Readings above 1.5 inH2O on AHU_1's
          duct static pressure
SELECT count(*)
FROM samples
WHERE timeseries_id = '52422584-9894-50bd-
          9696-2fbb8b062518'
  AND value > 1.5;
-- expect: 8336
```

`ts` is a big number. Tomorrow turns it into a date.

Try it

1. What was the coldest outside air reading?

2. Is AHU_1's supply fan status ever anything other than 0 or 1?

Answers

```
SELECT ts, value FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
ORDER BY value ASC LIMIT 1;
-- expect: 1773740400 | 17.8308
```

```
SELECT DISTINCT value FROM samples
WHERE timeseries_id = '9ebf80dd-7821-510b-
        b2c0-cd0151e093c9'
ORDER BY value;
-- expect rows: 2
```

Only 0 and 1. Status points are numbers in a historian, not booleans.

Why it matters

Pasting UUIDs by hand doesn't scale. AHU_1 alone has 83 points, and a campus has tens of thousands. Day 6 puts the point list in a table, and Unit 4 replaces it with a SPARQL query: "give me the duct static pressure sensor of every AHU" → UUIDs.

Day 3 — Time: epoch seconds, `datetime()`, date ranges

Goal: turn `ts` into readable dates, and select a day or a month.

Read

`ts` is **Unix epoch seconds**: seconds since 1970-01-01 00:00:00 UTC. `1773621600` is 2026-03-16 00:40 UTC. Historians store time like this because integers sort fast, compare exactly and have no time zone ambiguity. Everything in this database is **UTC**.

SQLite's date functions read epoch seconds when you add the `'unixepoch'` modifier:

Function	Result for `1773621600`
`datetime(ts, 'unixepoch')`	`2026-03-16 00:40:00`
`date(ts, 'unixepoch')`	`2026-03-16`
`strftime('%H', ts, 'unixepoch')`	`00` (hour)
`strftime('%w', ts, 'unixepoch')`	`1` (day of week, 0 = Sunday)

Function	Result for `1773621600`
`unixepoch('2026-07-01')`	`1782864000` (text → epoch seconds)

To select a time range, compare `ts` to epoch numbers. Write the range **half-open**, `start <= ts < end`, so consecutive days never share a reading.

Don't filter with `date(ts, 'unixepoch') = '2026-07-01'`. It gives the same rows, but SQLite has to compute a date for every row instead of jumping to the range. Day 12 shows the difference.

Run

```
SELECT datetime(ts, 'unixepoch') AS
        time_utc, value
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
ORDER BY ts
LIMIT 3;
-- expect: 2026-03-16 00:40:00 | 69.0327


-- First and last timestamp in the whole
        database
SELECT datetime(min(ts), 'unixepoch'),
        datetime(max(ts), 'unixepoch')
        FROM samples;
-- expect: 2026-03-16 00:40:00 | 2026-07-
        17 09:55:00
```

```
-- All outside air readings on 1 July 2026
        (UTC)
SELECT count(*)
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
  AND ts >= unixepoch('2026-07-01')
  AND ts <  unixepoch('2026-07-02');
-- expect: 288
```

288 = 24 hours × 12 readings an hour: a full day of 5-minute data.

Try it

1. When was the hottest outside air reading, as a date and time?
2. How many readings does the first day, 2026-03-16, have, and why isn't it 288?

Answers

```
SELECT datetime(ts, 'unixepoch'), value
        FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
ORDER BY value DESC LIMIT 1;
-- expect: 2026-07-14 21:40:00 | 98.8608
```

```
SELECT count(*) FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
  AND ts >= unixepoch('2026-03-16') AND ts
        < unixepoch('2026-03-17');
-- expect: 280
```

The data starts at 00:40, not midnight, so the first eight readings of that day were never logged.

Why it matters

Building people think in local time ("the fan should start at 6 am"). Databases should store UTC. Converting at the edges, and noticing when a "6 am start" shows up at 11:00 UTC, is a daily skill. The RDF model will record the site's time zone so code doesn't have to guess.

Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM

Goal: summarize a sensor in one row, and turn a status point into run hours.

Read

Aggregate functions fold many rows into one value:

Function	Notes
`count(*)`	rows
`count(value)`	rows where `value` is not NULL
`min(value)`, `max(value)`	ignore NULLs
`avg(value)`	mean of the non-NULL values
`sum(value)`	total
`round(x, n)`	n decimal places, for reading, not storage

Without `GROUP BY` (tomorrow), an aggregate query returns exactly one row.

A useful trick: a status point is 1 when running and 0 when off. With one reading every 5 minutes, `sum(status) × 5 / 60` is the **run hours**. Use `60.0`, not `60`: integer division in SQL drops the remainder.

Run

```
-- AHU_1 outside air temperature, whole
        dataset
SELECT count(*) AS n,
       round(min(value), 2) AS min_f,
       round(max(value), 2) AS max_f,
       round(avg(value), 3) AS avg_f
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48';
-- expect: 35536 | 17.83 | 98.86 | 61.276


-- The same for one day
SELECT round(min(value), 1),
        round(max(value), 1),
        round(avg(value), 1)
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
        9069-3b0f17d49c48'
  AND ts >= unixepoch('2026-07-01') AND ts
        < unixepoch('2026-07-02');
-- expect: 79.9 | 97.1 | 88.3
```

```sql
-- AHU_1 supply fan run hours
SELECT round(sum(value) * 5 / 60.0, 1) AS
         run_hours
FROM samples
WHERE timeseries_id = '9ebf80dd-7821-510b-
         b2c0-cd0151e093c9';
-- expect: 1574.4
```

The dataset covers about 2,961 hours, so the fan ran a bit over half the time.

Try it

1. What are the average duct static pressure and its average setpoint on AHU_1? Which is bigger?
2. What fraction of the time was AHU_1's fan running? (Hint: the average of a 0/1 column is the fraction.)

Answers

```sql
SELECT round(avg(value), 3) FROM samples
WHERE timeseries_id = '52422584-9894-50bd-
         9696-2fbb8b062518';
-- expect: 0.783
```

```sql
SELECT round(avg(value), 3) FROM samples
WHERE timeseries_id = '761c2b61-4982-5157-
         bc66-2010cf6e2a47';
-- expect: 1.366
```

```
SELECT round(avg(value), 3) FROM samples
WHERE timeseries_id = '9ebf80dd-7821-510b-
         b2c0-cd0151e093c9';
-- expect: 0.532
```

The average static (0.78) is far below the average setpoint (1.37). But that includes all the hours the fan was off, when static is near zero. Day 8 asks the fair question: what is static *while the fan runs*?

Why it matters

Averages over everything hide what matters. A sensor reading 0 °F because the unit is off pulls the average down just like a cold day does. Most of fault detection is asking "the average **of what, when**?" That's filtering (Days 8–9) and gating (week 3).

Day 5 — GROUP BY and HAVING: per day, per month, per series

Goal: one summary row per group, and keep only the groups you care about.

Read

`GROUP BY` splits rows into groups. Each aggregate is then computed once per group:

```
SELECT group_columns, aggregates
FROM ...
WHERE   row filter           -- before
grouping
GROUP BY group_columns
HAVING  group filter          -- after
grouping, can use aggregates
ORDER BY ...
```

`WHERE` throws away rows before they're grouped. `HAVING` throws away whole groups after. You can't put `avg(...)` in a `WHERE`, because no group exists yet at that point.

You can group by an expression, such as the date, so `GROUP BY date(ts, 'unixepoch')` gives one row per day. SQLite lets you use a column alias (`AS day`) in `GROUP BY` and `ORDER BY`.

Run

```sql
-- Monthly average outside air temperature
SELECT strftime('%Y-%m', ts, 'unixepoch')
         AS month,
       round(avg(value), 1) AS avg_f
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
         9069-3b0f17d49c48'
GROUP BY month
ORDER BY month;
-- expect: 2026-03 | 41.5


-- The hottest day on average
SELECT date(ts, 'unixepoch') AS day,
         round(avg(value), 1) AS avg_f
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
         9069-3b0f17d49c48'
GROUP BY day
ORDER BY avg_f DESC
LIMIT 1;
-- expect: 2026-07-02 | 88.3


-- Days that don't have a full 288
         readings
SELECT date(ts, 'unixepoch') AS day,
         count(*) AS n
FROM samples
WHERE timeseries_id = 'bf422925-1e2d-598d-
         9069-3b0f17d49c48'
GROUP BY day
HAVING count(*) < 288
ORDER BY day;
-- expect rows: 2
```

The two short days are the first (2026-03-16, starts at 00:40) and the last (2026-07-17, ends at 09:55). There are no gaps in between. Real historians usually aren't this tidy, so checking is always worth it.

Try it

1. Readings per series: does every one of the 118 series have the same count?
2. Daily run hours of AHU_1's supply fan. Which day ran longest?

Answers

```
SELECT n AS readings, count(*) AS series
FROM (SELECT timeseries_id, count(*) AS n
          FROM samples GROUP BY
          timeseries_id)
GROUP BY n;
-- expect: 35536 | 118
```

One row back means one count for everyone: all 118 series have 35,536 readings. (A query in a `FROM` like this is a **subquery**: a table made on the fly.)

```
SELECT date(ts, 'unixepoch') AS day,
         round(sum(value) * 5 / 60.0, 1)
         AS run_hours
FROM samples
WHERE timeseries_id = '9ebf80dd-7821-510b-
         b2c0-cd0151e093c9'
GROUP BY day
ORDER BY run_hours DESC, day
LIMIT 3;
-- expect rows: 3
```

Several days ran all 24 hours. Is that normal for an office AHU? That's a question for the building's operators, and it's the kind of finding retro-commissioning starts from.

Week 1 challenge

Using only the four UUIDs from Day 2, write **one query** that reports, for each month: the average outside air temperature, AHU_1's fan run hours, and the number of readings where duct static was above 1.5 inH2O. (Hint: you can't do it cleanly yet. Try, notice what makes it awkward, and keep the attempt for Day 9.)

Reading — Week 1

The lessons first, then these for exact behavior.

- DB Browser for SQLite
- SQLite SELECT
- SQLite date and time functions
- SQLite aggregate functions
- RFC 9562: UUIDs (version 5 = name-based)